ADAPT-VQE with Best Arm Identification (BAI) — Implementation Guide

Overview

This project implements the **Successive Elimination (SE)** algorithm for generator selection in **ADAPT-VQE**, as described in *"Quantum Gambling: Best-Arm Strategies for Generator Selection in Adaptive Variational Algorithms"* (Huang & Izmaylov, 2025). The key idea is to reformulate generator selection as a **Best Arm Identification (BAI)** problem, where each generator in the operator pool is an "arm" and its energy gradient is the "reward." By adaptively allocating measurements and discarding weak candidates early, the approach reduces measurement overhead by 60–92% compared to the naïve fixed-precision baseline.

Two main scripts implement this:

- **`adapt_vqe_exact_bai_scipy_minimization_multi_params.py`** — BAI with multiple SE rounds (full Successive Elimination)
- **`adapt_vqe_exact_estimates.py`** — Single-round exact gradient estimation (naïve baseline)

How the Scripts Work

Common Architecture

Both scripts follow the same ADAPT-VQE loop:

1. **Initialize** with the Hartree–Fock reference state
2. **Estimate gradients** `g_i = <ψ|[H, G_i]|ψ>` for each generator in the pool
3. **Select the best generator** (largest |g_i|)
4. **Append** the selected generator to the ansatz
5. **Re-optimize all parameters** globally via L-BFGS-B (`sparse_multi_parameter_energy_optimization`)
6. **Repeat** until convergence (gradient below threshold or energy within chemical accuracy of exact)

The gradient estimation uses:

- **QWC (qubit-wise commuting) decomposition** of the commutator `[H, G_i]` into measurable fragments
- **Exact statevector** computation of fragment expectation values and variances
- **Simulated sampling** from normal distributions `N(exact_mean, exact_variance)` to model shot noise

`adapt_vqe_exact_bai_scipy_minimization_multi_params.py` (SE Strategy)

This implements the **Successive Elimination** BAI algorithm from the paper:

- **Multiple rounds** (up to `max_rounds=10`) of gradient estimation with increasing precision
- **Precision schedule**: `ε_r = x - (x - target_accuracy) * r / 10`, starting coarse and refining
- **Elimination rule** (Eq. 3 in paper): After each round, arms where `|g_i| + R_r < M - R_r` are eliminated, where `M = max|g_i|` and `R_r = y * ε_r` is the confidence radius
- **Early stopping**: If only one arm remains, selection is complete
- Gradients are recomputed each round for the surviving active arms only

Key function: `bai_find_the_best_arm_exact_with_statevector()` — runs the SE loop with parallel gradient computation via `multiprocessing.Pool`.

`adapt_vqe_exact_estimates.py` (Naïve Baseline)

This implements the **fixed-precision baseline**:

- **Single round** of gradient estimation at the target accuracy for all generators
- No elimination — all pool operators are measured to the same precision
- Directly selects the arm with the largest estimated gradient

Key function: `bai_find_the_best_arm_exact_with_statevector()` — same interface but runs only one round (no elimination loop).

Gradient Computation Pipeline

Both scripts share the same gradient computation logic:

```
compute_exact_commutator_gradient_with_statevector()
  → get_commutator_qubit(H, G_i)          # Compute [H, G_i]
  → qwc_decomposition(commutator)          # Decompose into QWC groups
  → For each QWC group:
      → compute_pauli_expectation_fast()    # Exact <ψ|P|ψ> via Qiskit
      → Compute fragment mean and variance
      → Determine shots: ceil(variance / ε²)
      → Sample from N(mean, variance) to simulate measurements
  → Sum fragment contributions → estimated gradient, variance, N_est, t
```

Parameter Optimization

After selecting a generator, both scripts use `sparse_multi_parameter_energy_optimization()` from `sparse_energy_calculation.py`, which:

- Applies all ansatz operators sequentially via sparse matrix exponentials (`expm_multiply`)
- Optimizes all parameters simultaneously using L-BFGS-B
- Falls back to grid search if no improvement is found

Command-Line Arguments

Both scripts take the same 9 arguments:

```
python <script>.py <mol_file> <mol> <n_qubits> <n_electrons> <pool_type
```

Argument	Description	Example
`mol_file`	Hamiltonian binary file in `ham_lib/`	`h2o_fer.bin`
`mol`	Molecule name (used for output directory)	`h2o`
`n_qubits`	Number of qubits	`14`
`n_electrons`	Number of electrons	`10`
`pool_type`	Operator pool: `uccsd_pool`, `qubit_pool`, `qubit_excitation`	`uccsd_pool`
`shots`	Base shots per round	`1024`
`x`	Initial precision / coarse accuracy (ε_0 for SE)	`0.005`
`y`	Confidence radius multiplier (d_r in paper)	`8`
`target_accuracy`	Final target precision (ε in paper)	`0.001`

Typical configurations (from `notes.txt`):

- UCCSD/QE pools: `x=0.005, y=8, target_accuracy=0.001`
- Qubit pool: `x=0.001, y=8, target_accuracy=0.0005`

Running on the Trillium Cluster (Compute Canada)

Prerequisites

1. **Python virtual environment** set up on the cluster (see setup instructions below)
2. **`notes.txt`** in the project root, containing one command per line (see format above)
3. **Hamiltonian files** in `ham_lib/` (e.g., `h2o_fer.bin`)
4. **Output directories** for each molecule (e.g., `mkdir h2o`)

Python Environment Setup

There are two approaches: **pip with venv** (traditional) or **uv** (modern, faster). Both work on Compute Canada clusters.

Option A: Traditional `venv` + `pip` (Trillium)

This is what `submit_all_commands_1.sh` expects. The environment lives on `/scratch` for performance.

```
# 1. SSH into Trillium
ssh <username>@trillium.computecanada.ca

# 2. Load required modules
#    Trillium uses StdEnv/2023. The symengine module is needed for Qisk
module load StdEnv/2023 gcc python/3.11 symengine/0.11.2

# 3. Create the virtual environment on /scratch (faster I/O than /home)
python -m venv /scratch/$USER/adapt_vqe_env
```

```
# 4. Activate it
source /scratch/$USER/adapt_vqe_env/bin/activate

# 5. Upgrade pip first (cluster pip can be outdated)
pip install --upgrade pip

# 6. Install core scientific packages first (these have C extensions)
pip install numpy>=1.21.0 scipy>=1.8.0

# 7. Install quantum chemistry packages
pip install openfermion>=1.0.0 pyscf>=2.0.0 openfermionpyscf>=0.5.0

# 8. Install Qiskit (may take a few minutes)
pip install qiskit>=0.40.0 qiskit-aer>=0.12.0

# 9. Install remaining dependencies
pip install matplotlib>=3.5.0 pandas>=1.3.0 psutil>=5.8.0

# 10. Verify the installation
python -c "
import numpy as np
import scipy
import qiskit
import openfermion
import pyscf
import psutil
print('NumPy:', np.__version__)
print('SciPy:', scipy.__version__)
print('Qiskit:', qiskit.__version__)
print('OpenFermion:', openfermion.__version__)
print('PySCF:', pyscf.__version__)
print('All imports successful!')
"
```

Alternatively, install everything at once from `requirements.txt`:

```
module load StdEnv/2023 gcc python/3.11 symengine/0.11.2
python -m venv /scratch/$USER/adapt_vqe_env
source /scratch/$USER/adapt_vqe_env/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

> **Important**: The `submit_all_commands_1.sh` script expects the venv at `/scratch/rick0317/adapt_vqe_env`. Update the `VENV_PATH` variable in the script if your username differs:
>
> `VENV_PATH="/scratch/$USER/adapt_vqe_env"`

Option B: Traditional `venv` + `pip` (General Cluster / Narval / Cedar / Graham)

This is what `submit_all_commands_10x.sh` expects. The environment lives in `$HOME`.

```
# 1. SSH into the cluster
```

```
ssh <username>@narval.computecanada.ca  # or cedar, graham, etc.

# 2. Load modules (varies by cluster)
module load python
module load conda3  # Some clusters need this for symengine/C deps

# 3. Create venv in home directory
python -m venv ~/adapt-vqe-env

# 4. Activate and install
source ~/adapt-vqe-env/bin/activate
pip install --upgrade pip
pip install -r requirements.txt

# 5. Verify
python -c "import numpy, scipy, qiskit, openfermion; print('OK')"
```

Option C: Using `uv` (Modern, Faster)

`uv` is significantly faster than `pip` for dependency resolution and installation. This approach uses `pyproject.toml` instead of `requirements.txt`.

```
# 1. SSH into the cluster
ssh <username>@trillium.computecanada.ca

# 2. Load Python module
module load StdEnv/2023 gcc python/3.11 symengine/0.11.2

# 3. Install uv (one-time setup)
curl -LsSf https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.cargo/bin:$PATH"

# 4. Add to your .bashrc so it persists across sessions
echo 'export PATH="$HOME/.cargo/bin:$PATH"' >> ~/.bashrc

# 5. Navigate to the project directory and sync
cd /path/to/ADAPT-VQE-BAI
uv sync

# 6. Verify
uv run python -c "import numpy, scipy, qiskit, openfermion; print('OK')

# 7. Run scripts with uv
uv run python adapt_vqe_exact_bai_scipy_minimization_multi_params.py \
    h2o_fer.bin h2o 14 10 uccsd_pool 1024 0.005 8 0.001
```

> **Note**: If using `uv`, you need to modify the submit scripts to use `uv run python` instead of `python`. See `docs/CLUSTER_DEPLOYMENT.md` for details.

Troubleshooting Environment Issues

`symengine` errors with Qiskit

On Compute Canada clusters, Qiskit requires the `symengine` module. If you see errors like `ImportError: libsymengine.so: cannot open shared object file`:

```
# On Trillium:
module load symengine/0.11.2

# Add to your job script (already included in submit_all_commands_1.sh)
module load StdEnv/2023 gcc python/3.11 symengine/0.11.2
```

`pip install` fails for PySCF or OpenFermion

These packages compile C/Fortran extensions. Make sure `gcc` is loaded:

```
module load StdEnv/2023 gcc
pip install pyscf openfermion
```

Memory errors during install

On login nodes, memory is limited. Use an interactive job for installation:

```
salloc --time=1:00:00 --mem=8G --cpus-per-task=4 --account=rrg-izmaylov
# Then run the pip install commands inside the allocation
```

Virtual environment not found at job runtime

The SLURM job runs in a clean shell. Ensure:

1. The venv path is absolute (not relative)
2. The venv is on a filesystem accessible from compute nodes (`/scratch` or `$HOME`, **not** `/tmp`)
3. Modules are loaded **before** activating the venv in the job script

Testing the environment before submitting jobs

Run a quick interactive test:

```
salloc --time=0:30:00 --mem=4G --cpus-per-task=4 --account=rrg-izmaylov
module load StdEnv/2023 gcc python/3.11 symengine/0.11.2
source /scratch/$USER/adapt_vqe_env/bin/activate
cd /path/to/ADAPT-VQE-BAI

# Quick smoke test with the smallest molecule
python -u adapt_vqe_exact_estimates.py h4_fer.bin h4 8 4 qubit_pool 102
```

Dependencies Summary

Package	Version	Purpose
`openfermion`	>= 1.0.0	Fermionic operator algebra, Jordan-Wigner transform
`openfermionpyscf`	>= 0.5.0	Interface between OpenFermion and PySCF

`pyscf`	>= 2.0.0	Molecular integrals, Hamiltonian generation
`qiskit`	>= 0.40.0	Quantum circuit simulation, SparsePauliOp
`qiskit-aer`	>= 0.12.0	Statevector simulator backend
`numpy`	>= 1.21.0	Array operations, linear algebra
`scipy`	>= 1.8.0	Sparse matrices, L-BFGS-B optimizer, `expm_multiply`
`matplotlib`	>= 3.5.0	Plotting (for analysis, not needed at runtime)
`pandas`	>= 1.3.0	CSV output handling
`psutil`	>= 5.8.0	Memory usage tracking (optional but recommended)

`submit_all_commands_1.sh` — Trillium (Large Nodes)

Designed for **Trillium's large-memory nodes** (192 CPUs per node).

```
# From the project directory on Trillium:
bash submit_all_commands_1.sh
```

SLURM configuration per job:

- Account: `rrg-izmaylov`
- 1 node, 1 task, 192 CPUs
- Wall time: 23h 30m
- Submits **5 runs** per command in `notes.txt`
- Uses modules: `StdEnv/2023 gcc python/3.11 symengine/0.11.2`
- Venv: `/scratch/rick0317/adapt_vqe_env`
- Thread settings: `OMP/OPENBLAS/MKL/NUMEXPR_NUM_THREADS=192`

`submit_all_commands_10x.sh` — General Cluster

Designed for standard cluster nodes with fewer resources.

```
# From the project directory:
bash submit_all_commands_10x.sh
```

SLURM configuration per job:

- Account: `rrg-izmaylov`
- 1 node, 40 tasks
- Wall time: 10h 30m
- Submits **20 runs** per command in `notes.txt`
- Uses modules: `python`, `conda3`
- Venv: `~/adapt-vqe-env`
- Thread settings: `OMP/OPENBLAS/MKL_NUM_THREADS=4`

Monitoring Jobs

```
# Check all your jobs
squeue -u $USER

# Filter by molecule
squeue -u $USER | grep h2o
```

```
# Cancel all jobs
scancel -u $USER
```

Output Files

Each job produces:

- **CSV results**: `<mol>/adapt_vqe_bai_<pool>_results_<timestamp>_<x>_<y>.csv` (SE) or `<mol>/adapt_vqe_<pool>_results_<timestamp>_exact_estimates.csv` (baseline)
- **Log file**: `<script>_<mol>_<pool>_run<N>_<jobid>.log`
- **Results directory**: `results_<script>_<mol>_<pool>_run<N>_<timestamp>_<jobid>/` (copies of all CSVs and logs)
- **SLURM output**: `<script>_<mol>_<pool>_run<N>_<jobid>.out` and `.err`

Preparing a New Molecule

1. Generate the fermionic Hamiltonian and save as a pickle file in `ham_lib/`
2. Create the output directory: `mkdir <mol_name>`
3. Add a line to `notes.txt` with the appropriate parameters
4. Run the submission script

Relationship to the Paper

Paper Concept	Implementation
Generator pool $A = \{G_i\}$	`generator_pool` from `get_generator_pool()`
Gradient $g_i = \langle\psi\|[H, G_i]\|\psi\rangle$	`compute_exact_commutator_gradient_with_statevector()`
QWC fragmentation $[H, G_i] = \Sigma A_n$	`qwc_decomposition()` from `utils.decomposition`
Fragment variance $Var(A_n)$	Computed as `Σ(c²)(1 - <P>²)` per QWC group
Shots per fragment $M_n(\varepsilon) = Var/\varepsilon^2$	`shots_per_fragment = ceil(variance / radius²)`
SE round precision ε_r	`accuracy = x - (x - target_accuracy) * r / 10`
Confidence radius $R_r = d_r \cdot \varepsilon_r$	`radius = accuracy * y`
Elimination rule (Eq. 3)	`if abs(means[i]) + radius >= max_mean - radius: keep`
VQE re-optimization (L-BFGS-B)	`sparse_multi_parameter_energy_optimization()`
Naïve baseline (fixed ε)	`adapt_vqe_exact_estimates.py` (single round, no elimination)